Prescripts and postscripts

SnapCenter Plug-in for VMware vSphere

Ronya Robinson, Madhulika Kola
May 06, 2021

This PDF was generated from https://docs.netapp.com/us-en/sc-plugin-vmware-vsphere/scpivs44_prescripts_and_postscripts.html on June 24, 2021. Always check docs.netapp.com for the latest.

Table of Contents

Prescripts and postscripts . . . 1
- Supported script types . . . 1
- Script path location . . . 1
- Where to specify scripts . . . 1
- When scripts are executed . . . 1
- Environment variables passed to scripts . . . 2
- Script timeouts . . . 2
- Example PERL script #1 . . . 3
- Example PERL script #2 . . . 3
- Example shell script . . . 3

Prescripts and postscripts

You can use custom prescripts and postscripts as part of your data protection operations. These scripts enable automation either before your data protection job or after. For example, you might include a script that automatically notifies you of data protection job failures or warnings. Before you set up your prescripts and postscripts, you should understand some of the requirements for creating these scripts.

Supported script types

Perl and shell scripts are supported.
Shell scripts must start with `!/bin/bash`. (`!/bin/sh` is not supported.)

Script path location

Prescripts and postscripts are run by the SnapCenter Plug-in for VMware vSphere. Therefore, the scripts must be located in the SnapCenter Plug-in for VMware vSphere OVA, with executable permissions.

For example:
* A PERL script path might be `/support/support/script.pl`
* A shell script path might be `/support/support/script.sh`

The script path is validated at the time the script is executed.

Where to specify scripts

Scripts are specified in backup policies. When a backup job is started, the policy automatically associates the script with the resources being backed up.

To specify multiple scripts, press **Enter** after each script path to list each script on a separate line. Semicolons (;) are not allowed. You can specify multiple prescripts and multiple postscripts. A single script can be coded as both a prescript and a postscript and can call other scripts.

When scripts are executed

Scripts are executed according to the value set for BACKUP_PHASE.

- BACKUP_PHASE=PRE_BACKUP

  Prescripts are executed in the PRE_BACKUP phase of the operation.

  If a prescript fails, the backup completes successfully, and a warning message is sent.

- BACKUP_PHASE=POST_BACKUP or BACKUP_PHASE=FAILED_BACKUP

  Postscripts are executed in the POST_BACKUP phase of the operation after the backup completes successfully or in the FAILED_BACKUP phase if the backup does not complete successfully.

If a postscript fails, the backup completes successfully, and a warning message is sent.

Check the following to verify that the script values are populated:
* For PERL scripts: `/support/support/log_env.log`
* For shell scripts: `/support/support/log_file.log`

Environment variables passed to scripts

You can use the environment variables shown in the following table in scripts.

Environment variable	Description
`BACKUP_NAME`	Name of the backup. Variable passed in postscripts only.
`BACKUP_DATE`	Date of the backup, in the format `yyyymmdd` Variable passed in postscripts only.
`BACKUP_TIME`	Time of the backup, in the format `hhmmss` Variable passed in postscripts only.
`BACKUP_PHASE`	The phase of the backup in which you want the script to run. Valid values are: `PRE_BACKUP, POST_BACKUP, and FAILED_BACKUP`. Variable passed in prescripts and postscripts.
`STORAGE_SNAPSHOTS`	The number of storage snapshots in the backup. Variable passed in postscripts only.
`STORAGE_SNAPSHOT.#`	One of the defined storage snapshots, in the following format: `<filer>:/vol/<volume>:<ONTAP-snapshot-name>` Variable passed in postscripts only.
`VIRTUAL_MACHINES`	The number of VMs in the backup. Variable passed in prescripts and postscripts.

`VIRTUAL_MACHINE.#`	One of the defined virtual machines, in the following format:
	`<VM name>[vertical bar]<VM UUID>[vertical bar]<power-state>[vertical bar]<VM snapshot>[vertical bar]<ip-addresses>`
	`<power-state> has the values POWERED_ON, POWERED_OFF, or SUSPENDED`
	`<VM snapshot>` has the values `true` or `false`
	Variable passed in prescripts and postscripts.

Script timeouts

The timeout for backup scripts is 15 minutes and cannot be modified.

Example PERL script #1

The following example PERL script prints the environmental variables when a backup is run.

```
#!/usr/bin/perl
use warnings;
use strict;
my $argnum;
my $logfile = '/support/support/log_env.log';
open (FH, '>>', $logfile) or die $!;
foreach (sort keys %ENV) {
print FH "$_ = $ENV{$_}\n";
}
print FH "=========\n";
close (FH);
```

Example PERL script #2

The following example prints information about the backup.

```
#!/usr/bin/perl
use warnings;
use strict;

my $argnum;
my $logfile = '/support/support/log_env.log';
open (FH, '>>', $logfile) or die $!;

print FH "BACKUP_PHASE is $ENV{'BACKUP_PHASE'}\n";
print FH "Backup name $ENV{'BACKUP_NAME'}\n";
print FH "Virtual Machine $ENV{'VIRTUAL_MACHINES'}\n";
print FH "VIRTUAL_MACHINE # is $ENV{'VIRTUAL_MACHINE.1'}\n";
print FH "BACKUP_DATE is $ENV{'BACKUP_DATE'}\n";
print FH "BACKUP_TIME is $ENV{'BACKUP_TIME'}\n";
print FH "STORAGE_SNAPSHOTS is $ENV{'STORAGE_SNAPSHOTS'}\n";
print FH "STORAGE_SNAPSHOT # is $ENV{'STORAGE_SNAPSHOT.1'}\n";

print FH "PWD is $ENV{'PWD'}\n";
print FH "INVOCATION_ID is $ENV{'INVOCATION_ID'}\n";

print FH "=========\n";
close (FH);
```

Example shell script

```
================================================
#!/bin/bash
echo Stage $BACKUP_NAME >> /support/support/log_file.log
env >> /support/support/log_file.log
================================================
```

Copyright Information

Copyright © 2021 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.